Google

JAXBench: Benchmarking Autonomous TPU Kernel Optimization

Arya Tschand[*,1,4], **Charles Hong**[*,2,4], **Julian Walker**[3], **Nina Cai**[4], **Shangkun Wang**[4], **Suvinay Subramanian**[4], **Sundar Dev**[4], **Vijay Janapa Reddi**[1], **Amir Yazdanbakhsh**[3] **and Sethu Sankaran**[4]
[*]Equal contributions, work done while at Google, [1]Harvard University, [2]UC Berkeley, [3]Google DeepMind, [4]Google

Code: https://github.com/AI-Hypercomputer/accelerator-agents/tree/main/JAXBench

Rigorous benchmarks have driven progress in autonomous GPU kernel performance optimization by establishing a shared target to hillclimb on, but no equivalent exists for TPUs. We present JAXBench, a TPU-native benchmark suite for AI-generated kernel optimization on Google Cloud TPUs. JAXBench comprises 50 JAX workloads that are both *relevant* and provide *headroom* for optimization. We extract 17 production ML operators from architectures in the public MaxText library such as Llama-3.1, DeepSeek-V3, Mixtral, Mamba-2, and AlphaFold2, and translate 33 operators from KernelBench that are validated for correctness and set with new problem sizes that achieve high TPU v6e MXU utilization. Eight of the 17 production operators ship with hand-optimized Pallas kernels from the public Tokamax library and block-size tuned to establish an expert upper-bound baseline. We evaluate four feedback-driven methods on generating candidate Pallas kernels for JAXBench. Across the full suite with Gemini 3 Flash, we find that target-specific context matters more than model scale on a sparsely-documented DSL like Pallas. Conditioning on curated TPU documentation raises per-sample correctness from 5.8% to 37.3% and solves 48 of 50 benchmarks at a 1.28× geomean speedup. Search structure yields significant gains once correctness is achieved, with Autocomp's beam-search pipeline reaching a 1.36× geomean speedup over XLA. On the 8 hand-tuned kernels, Autocomp reaches 1.60× geomean over XLA, recovering most of the 2.08× Tokamax upper bound but trailing on the specialized paged and ragged attention operators. High-quality TPU kernel optimization remains a challenging task, and we release the JAXBench benchmark, evaluation harness, and baseline results to support open source contributions.

1. Introduction

Efficient kernel implementations are a bottleneck in achieving the full potential of hardware accelerators for machine learning. Libraries such as cuBLAS, CUTLASS, and Triton [25, 26] and specialized kernels for GPU [3] and TPU [10] have unlocked generation-over-generation performance gains, but each new model architecture, quantization scheme, and hardware revision demands fresh low-level implementations. To realize the performance gains enabled by architecture, kernels must be rapidly co-designed between the hardware-specific features and emerging workload characteristics.

This gap has motivated a fast-growing body of work on using language models to generate and optimize kernels automatically, ranging from one-shot completions to iterative agents with compile and profile feedback to evolutionary search over program space [2, 14, 16, 27]. Rigorous benchmarks have been central to this progress. KernelBench [20] standardized the evaluation protocol with 250 PyTorch workloads and has since been extended to Triton, CuTe, and other DSLs. TritonBench [15] specifically targets Triton generation with hardware-aware performance measurement on NVIDIA and AMD GPUs. FlashInfer-Bench [31] grounds evaluation in real LLM serving traces with expert-written FlashInfer kernels as references. Together, these benchmarks have driven rapid community progress on correctness and speedup against GPU baselines and have enabled meaningful comparison of methods from best-of-N sampling to reinforcement-learned coding agents.

Corresponding author(s): aryatschand@g.harvard.edu, sethusankaran@google.com

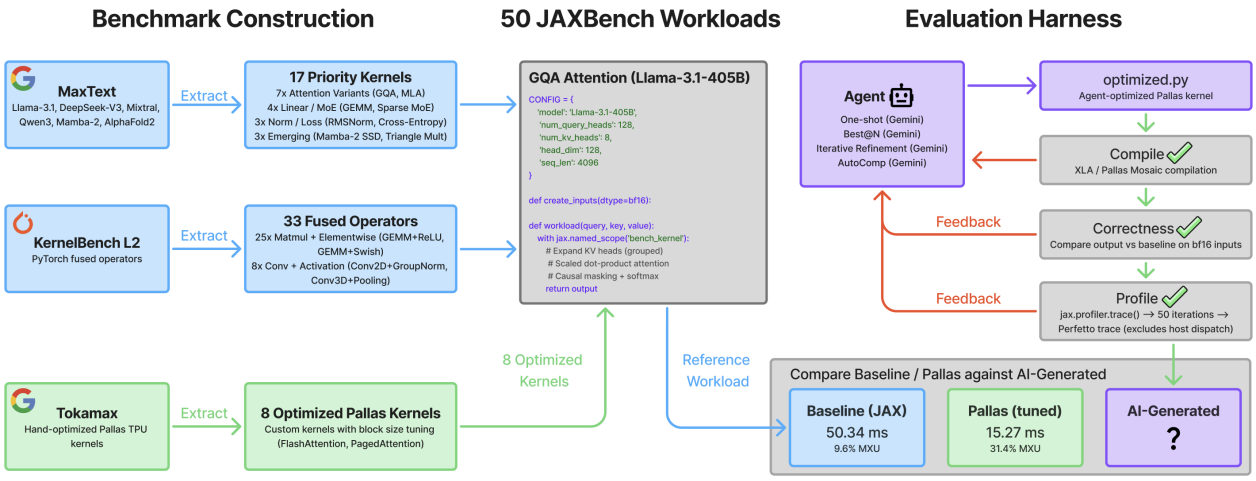


Figure 1 | **JAXBench overview: a TPU-native benchmark for evaluating AI-generated Pallas kernels against hand-tuned baselines on TPU v6e.** We construct 50 JAX reference workloads (blue) with 17 priority kernels from MaxText [4] LLMs and 33 fused operators adapted from KernelBench L2, all sized to saturate TPU v6e MXUs. We also extract 8 priority kernels of hand-tuned Pallas implementations (green) from Tokamax as expert upper bounds. The agent evaluation harness (purple) enables evaluation of any LLM-driven method to generate candidate Pallas kernels, which are compiled, correctness-checked on bf16 inputs, and profiled via `jax.profiler`, with feedback fed back and final kernels compared against both baselines.

However, *no analogous benchmark exists for TPU kernel optimization.* Google's Tensor Processing Units [11] differ fundamentally from GPUs. TPUs are sequential machines with wide SIMD vector registers (8×128 for 32-bit values on v6e) and dedicated 256×256 systolic Matrix Multiply Units (MXUs), rather than the massively parallel SIMT execution model of GPUs. Programming TPUs also requires a distinct software stack. JAX [1] programs compile through XLA, and low-level kernel authoring goes through Pallas, which lowers to the Mosaic backend rather than Triton. Pallas kernels must reason about TPU-specific concerns including VMEM/SMEM/HBM memory hierarchies, software pipelining with prefetch scheduling, block shape constraints, and the lexicographic grid traversal order that Mosaic enforces [8]. Pallas also appears in LLM training data orders of magnitude less frequently than CUDA or even Triton, so models that fluently write GPU kernels routinely hallucinate Pallas APIs, emit memory-space annotations that do not type-check, or violate systolic tiling constraints in ways that no amount of generic compile feedback can resolve. GPU benchmarks therefore cannot be directly applied because workloads, problem sizes, programming abstractions, and optimization strategies are all GPU-specific.

Recent concurrent work has addressed multi-platform evaluation. MultiKernelBench [29] extends KernelBench to CUDA, Huawei AscendC, and Google Pallas, finding that the best model achieves only 8.4–10.5% Pass@1 on Pallas tasks. However, MultiKernelBench does not target JAX, evaluates on TPU v2-8, translates PyTorch workloads at problem sizes too small to saturate the MXU, and does not include production LLM operators or optimized reference kernels. At small shapes, workloads are dominated by memory traffic and launch overhead rather than compute, so any measured speedup reflects bookkeeping rather than genuine algorithmic or scheduling improvement. A useful TPU benchmark must instead push workloads into the compute-bound regime where the MXU is binding and tiling, pipelining, and layout choices actually affect throughput. Table 1 summarizes how existing benchmarks position on these axes. There remains a need for a TPU-native benchmark that (i) uses contemporary TPU hardware, (ii) includes workloads representative of modern LLM training and inference, (iii) provides strong baselines through both XLA compilation and hand-optimized Pallas

Table 1 | Comparison of LLM kernel-generation benchmarks. JAXBENCH is the only suite targeting contemporary TPUs with production-scale workloads, hardware-saturating problem sizes, and expert-written Pallas reference kernels for a subset.

Benchmark	Target	# Tasks	Workload Source	Expert Refs	Sat. Sizes
KernelBench [20]	CUDA / GPU	250	PyTorch modules	—	Yes
TritonBench [15]	Triton / GPU	184	Real repos	Triton	Partial
FlashInfer-Bench [31]	CUDA / GPU	–	LLM serving traces	FlashInfer	Yes
MultiKernelBench [29]	Pallas / TPU v2-8	285	PyTorch modules	—	No
JAXBENCH (ours)	**Pallas / TPU v6e**	**50**	**Production LLMs + Standard JAX**	**Pallas**	**Yes**

kernels, and (iv) sizes problems to be compute-bound so optimization headroom is meaningful.

We introduce JAXBENCH, a benchmark suite of 50 JAX workloads designed specifically for TPU kernel optimization. Our contributions are as follows.

1. **TPU-native benchmark suite with 50 relevant workloads.** JAXBENCH is a benchmark of 17 production operators from real LLM architectures (Llama-3.1, DeepSeek-V3, Mixtral, Mamba-2, AlphaFold2) extracted from MaxText, and 33 fused operator sequences from KernelBench translated from PyTorch to JAX and set with problem sizes adjusted to achieve high TPU MXU utilization. For 8 priority kernels, we additionally provide expert-optimized Pallas TPU kernels from the upstream Tokamax library with tuned block sizes.
2. **Rigorous and reproducible evaluation harness.** Device-side profiling via `jax.profiler` Perfetto traces eliminates host dispatch overhead, yielding reproducible per-iteration kernel timings that can be used to easily evaluate any agent framework on JAXBENCH.
3. **Agent evaluation and failure modes.** We deploy JAXBENCH on TPU v6e (Trillium) and evaluate best-of-N, iterative feedback agent loops, TPU context-conditioned agent loops, and Autocomp [7] augmented with TPU architecture documentation. We also provide insights into failure modes and opportunities for further contributions in TPU-centric kernel optimization.

2. Methodology

2.1. Design Principles

JAXBENCH is built around three principles that together make the benchmark both practically relevant and technically rigorous for TPU kernel optimization.

Relevant workloads. The benchmark must evaluate performance on a diverse set of operators representative of real-world TPU customers. We include both well-studied operators where expert-optimized Pallas kernels already exist (e.g., flash attention, grouped-query attention) and emerging operators where no such Pallas kernels have been developed (e.g., Mamba-2 state space duality, AlphaFold2 triangle multiplication). This combination tests automated optimization methods across varying levels of prior human effort.

Headroom to optimize. Workloads and problem sizes are chosen so that the XLA-compiled baseline already achieves high TPU utilization, leaving kernel optimization to compete on genuine algorithmic and scheduling improvements rather than on closing artificial gaps. This choice reflects how TPUs

are actually used. Customers running production workloads tune problem dimensions to maximally utilize the hardware, and FLOPs utilization at the workload level is tightly correlated with FLOPs utilization of the underlying operations. A benchmark that evaluates kernels at undersized, memory-bound, or launch-overhead-bound shapes would not reflect the regime in which TPU users (and therefore TPU kernel optimizers) actually operate. We therefore size each workload to push the relevant operations toward the compute-bound regime, where the MXU is the binding resource and optimization headroom is meaningful. As shown in Figure 2a, matmul-heavy fused operators use dimensions such as $(4096, 8192) \times (8192, 8192)$ in bf16, achieving 60–95% MXU utilization, and priority kernels use production-scale dimensions from their source architectures. The Llama-3.1-70B GEMM workload, for example, runs at $8192 \times 8192 \times 28672$ and achieves ~79% MXU utilization. Problem size is itself an axis of optimization, since different shapes expose different opportunities and sufficiently small problems offer no headroom at all.

Reproducible baselines. Every workload is implemented in idiomatic JAX, compiled via XLA's `jax.jit`. This is the strongest reproducible baseline for TPU because XLA already performs aggressive whole-program optimization including operator fusion, buffer assignment, and layout optimization. For priority kernels, when available, we additionally provide Pallas-optimized variants that represent the current state of the art in hand-tuned TPU kernel performance, establishing an upper bound that automated methods should aspire to match.

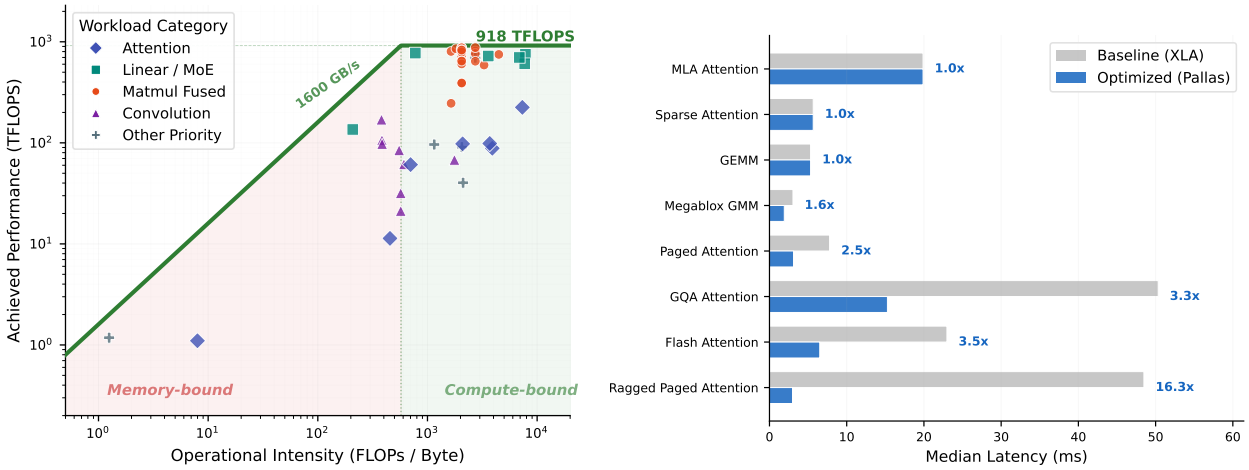
2.2. Workload Construction

Priority kernels (17 workloads). We extract 17 compute-critical operators from production LLM architectures as implemented in MaxText [4]. The set spans attention variants (flash [3], GQA [5], MLA [17], sparse/splash [9], flex, paged [13], and ragged paged), dense and sparse linear algebra (GEMM, SwiGLU MLP [21], sparse MoE [22], Megablox GMM, ragged dot), normalization and loss (RMSNorm [32], cross-entropy), and emerging architectures (RetNet retention [23], Mamba-2 SSD [6], AlphaFold2 triangle multiplication [12]). Each workload uses production-scale dimensions from its source model. The GQA attention workload, for example, uses Llama-3.1-405B dimensions with 128 query heads and 8 key-value heads at sequence length 4096.

Whenever available, we source optimized implementations by importing equivalent Pallas kernels from the upstream JAX Pallas ops library (Tokamax) [19], then tune their block size parameters via exhaustive grid search on TPU v6e. Eight such high quality kernels were available in the library. The tuning procedure evaluated 203 configurations total, yielding speedups of up to 2.79× (Megablox GMM) over the Pallas default parameters.

KernelBench fused operators (33 workloads). We translate a curated subset of 33 workloads from KernelBench Level 2 [20] from PyTorch to equivalent JAX. We focus on Level 2 because Level 1 consists of isolated single operators with no fusion structure, whereas Level 2 combines a matrix multiplication or convolution with elementwise operations (activations, normalization, pooling), directly representing the kernel fusion opportunities that are most relevant for TPU optimization. To select a non-redundant subset, we group all Level 2 tasks by their fusion signature, namely the (matmul or convolution) × (activation, normalization, pooling) operator class they instantiate, and retain one representative per class. This procedure deduplicates structurally identical tasks and yields the 33 workloads we include.

Translation is performed by prompting Gemini with each reference PyTorch module and asking for an idiomatic JAX equivalent. To confirm numerical correctness, we execute the original PyTorch



(a) TPU v6e roofline [30] with all 50 workloads.

(b) Tokamax Pallas kernel speedup over XLA.

Figure 2 | **JAXBench workloads are predominantly compute-bound on the TPU v6e roofline, and show headroom where existing Pallas kernel optimization can achieve up to a 16× speedup over the XLA baseline.** Achieved performance vs. operational intensity on a single TPU v6e chip (918 TFLOPS bf16, 1640 GB/s HBM) reveals that matmul-fused and linear/MoE kernels already reach 60–95% MXU utilization under XLA, while attention and elementwise kernels remain memory-bound. Hand-optimized Pallas kernels close this gap for the 8 priority workloads by tiling computation to reduce redundant HBM traffic, achieving speedups of up to 16.3× on Ragged Paged Attention and 3.5× on Flash Attention. Compute-bound workloads like GEMM show no improvement, confirming that XLA already saturates the MXU for high-OI operations and that optimization effort is best directed at memory-bound kernels.

reference and the generated JAX implementation on TPU with matched bf16 inputs and require their outputs to agree under `jnp.allclose` with atol = rtol = 10^{-2}, the standard bf16 tolerance. Any workload that fails this check is regenerated or repaired by hand before inclusion.

Many of the original KernelBench problem sizes are too small to saturate TPU MXUs, since the 256×256 systolic arrays require large matrix dimensions to achieve high utilization. Rather than apply a uniform scaling factor, we tune each workload's free dimensions independently by sweeping over candidate shapes and freezing the smallest configuration whose XLA baseline reaches at least 60% MXU utilization on TPU v6e. This per-operator procedure yields a heterogeneous set of shapes (e.g., batch size 4096 with feature dimensions 8192×8192 in bf16 for matmul-fused operators), all of which are compute-bound rather than launch-overhead-bound on TPU.

Workload interface. Every workload module exposes a standardized interface composed of a `CONFIG` dictionary of hyperparameters, a `create_inputs(dtype)` function that generates input tensors in bf16, and a `workload(*inputs)` function containing the computation to benchmark, which is annotated with `jax.named_scope` by the profiler harness for visibility. The benchmark runner JIT-compiles each workload via `jax.jit` and executes it under controlled timing (Section 2.3).

2.3. Timing and Profiling

Accurate kernel-level timing on TPUs requires device-side measurement. Wall-clock timing via `time.perf_counter()` with `block_until_ready()` includes host-side overhead from Python dispatch, JAX runtime scheduling, and synchronization. For short kernels (< 1 ms), this overhead can

constitute 10–20% of measured time.

We use `jax.profiler.trace()` to capture Perfetto-compatible traces and extract per-iteration execution times. For each workload, we (1) create inputs in bf16, (2) JIT-compile, (3) run 5 warmup iterations, (4) execute 50 timed iterations under the profiler, and (5) parse the trace to extract `jit_*()`-level device events. These events capture the total device-side execution time per iteration, regardless of how XLA or Pallas decomposes the computation internally (single fusion, multiple fusions, or a Pallas kernel call). We report the median over 50 iterations as the primary metric.

FLOP counting and utilization. We measure MXU utilization on TPU v6e and operational intensity of ragged paged attention [10] as

$$\text{MXU util}_{\text{v6e}} = \frac{\text{FLOPs}/t_{\text{median}}}{918 \times 10^{12}}, \qquad \text{OI}_{\text{ragged-attn}} = \frac{\text{FLOPs}_{\text{v6e}}}{\text{MemBW}_{\text{v6e}}} = \frac{G \sum_{i=1}^{B} L_i}{GB + \sum_{i=1}^{B} L_i} \tag{1}$$

where 918 TFLOPS is the TPU v6e single-chip bf16 peak, $G = H_q/H_{kv}$ is the GQA grouping factor, B is the number of sequences, and $\{L_i\}$ are the per-sequence KV lengths. The numerator counts attention FLOPs, which scale as $H_q \sum_i L_i$ since every query head attends over its full KV sequence. The denominator counts bytes read, with $H_q B$ from the per-sequence query tensors and $H_{kv} \sum_i L_i$ from the shared K and V cache (with constants for d_{head} and bytes-per-element absorbed into the proportionality). Dividing numerator and denominator by H_{kv} yields the form above. In the long-context limit $\sum_i L_i \gg GB$ the operational intensity therefore approaches G, reflecting that each KV token is reused by exactly G query heads and that this reuse caps achievable OI regardless of sequence length. For Llama-3.1-405B ($G = 16$), this upper bound of 16 FLOP/byte sits far below the v6e ridge point of OI ≈ 560, confirming that ragged paged attention is memory-bandwidth-bound at any context length. For the workloads, FLOPs are extracted from XLA's `cost_analysis()` on the compiled HLO program and bytes accessed are computed analogously for each operator class.

3. Results

3.1. Evaluation Protocol

We evaluate generated kernels along three axes. The first two are gating criteria, and the third is our reported performance metric. We additionally report one derived budget metric ($\text{fast}_1@N$) to summarize how quickly each method produces useful kernels.

1. Compilability. Does the generated code compile to a valid JAX/Pallas program? For Pallas kernels, this requires correct use of `BlockSpec`, `PrefetchScalarGridSpec`, memory space annotations, and block shape constraints specific to TPU.

2. Correctness. Does the generated kernel produce outputs that match the reference implementation? We check numerical agreement on randomized bf16 inputs under `jnp.allclose` with atol = rtol = 10^{-2}, the standard bf16 tolerance. Kernels that fail either gate contribute 1× speedup to the aggregate.

3. Speedup over XLA. Our primary performance metric is wall-clock speedup of the generated kernel over the XLA-compiled JAX baseline, measured with device-side profiler timing (Section 2.3). For the 8 priority kernels with hand-optimized Pallas implementations, we report performance against

those references separately as an upper-bound comparison rather than as the denominator of the aggregate speedup.

fast$_1$@N. As a sample-efficiency metric, fast$_1$@N reports the fraction of benchmarks whose best-so-far generated kernel already beats the XLA baseline after N cumulative samples emitted by the method. It complements geomean speedup by measuring breadth of coverage rather than magnitude of improvement at a given budget.

Evaluated methods. We evaluate four approaches.

(1) **Best-of-N generation.** Each sample is an independent one-shot completion conditioned on a TPU v6e preamble and the JAX source, and the best correct sample is selected. See Appendix B.

(2) **Iterative refinement.** Agent loops with access to compilation errors, correctness results, and profiler summaries as feedback between turns, following the paradigm of KernelBench [20] but adapted for JAX/Pallas. We run 18 independent chains of 8 turns each.

(3) **Iterative refinement with Autocomp context.** Identical to iterative refinement except that every turn receives Autocomp's agent context (hardware architecture summary, per-benchmark-selected Pallas API reference, selected code examples, and rules block) prepended to the preamble. This ablation isolates the effect of curated documentation from the effect of structured search.

(4) **Autocomp** [7]. An open-source LLM-driven kernel optimizer that ingests publicly available JAX Pallas and Cloud TPU documentation and distills it into four artifacts prepended to every agent prompt: a hardware-architecture summary, a curated Pallas API reference, annotated code examples, and correctness rules. We run a two-phase pipeline: a *translation phase* (4 beam-search iterations) rewrites the XLA baseline into a Pallas kernel, and an *optimization phase* (4 iterations) refines it for performance. Both phases use beam size 3 with 6 new candidates per beam element per iteration. We select Autocomp as our primary agent baseline because it is the most hardware-aware open method available. Other methods such as AlphaEvolve [18] and KernelEvolve [16] are not designed with TPU-specific context, and we leave their evaluation to future work.

Setup. We evaluate all four methods on TPU v6e (Trillium), starting from the XLA baseline. Best-of-N and both iterative variants use a fixed budget of 144 samples per benchmark. Autocomp uses a beam search of comparable total budget split across translation and optimization phases, and early-stops once its beam stops improving, so its actual sample count per benchmark can be lower than 144. For each method and benchmark we track best-so-far speedup over XLA at every sample, and aggregate across benchmarks by geometric mean.[1] All methods use Gemini 3 Flash [24] for the main evaluation on all 50 benchmarks (Section 3.2). We also show an ablation on a 5-kernel subset with Gemini 3.1 Pro (Section 3.4).

3.2. Baseline Comparison (Gemini 3 Flash)

We evaluate all four methods across the 50-workload JAXBench suite with Gemini 3 Flash (Table 2).

Speedup trajectory. Figure 3 shows best-so-far geomean speedup (left) and fast$_1$@N (right) as samples accumulate. The method ranking is stable almost from the first sample. Autocomp pulls ahead within the first dozen samples and holds the lead through the full budget, finishing at 1.36×

[1] We set the speedup floor to 1× to prevent correct but slow kernels from counting as worse than incorrect kernels.

Table 2 | Cross-method comparison on the full 50-benchmark JAXBench suite with Gemini 3 Flash. Per-benchmark best speedups are floored at 1× before aggregating; missing (incorrect) benchmarks count as 1×. Per-sample correctness is reported separately in Figure 4.

Method	Geomean speedup	Mean speedup	Per-benchmark correctness
Best-of-N	1.01×	1.02×	13/50
Iterative refinement	1.18×	1.50×	32/50
Iterative + context	1.28×	1.55×	**48/50**
Autocomp	**1.36**×	**1.70**×	45/50

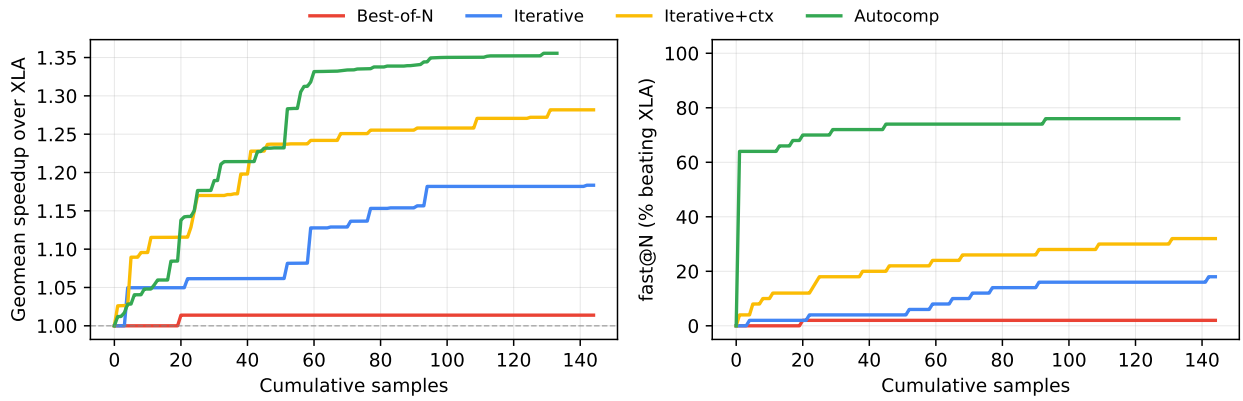


Figure 3 | Speedup and solved-rate trajectories on the full JAXBench suite (Gemini 3 Flash) as a function of cumulative samples. **Left:** geomean speedup over XLA (per-kernel best speedups floored at 1× before the geomean, so curves are monotonically non-decreasing). **Right:** $\text{fast}_1@N$, the fraction of benchmarks whose best-so-far sample already beats XLA at budget N.

geomean with 76% of benchmarks beating XLA. Iterative+context is the runner-up at 1.28× and 32% $\text{fast}_1@N$, a clear improvement over plain iterative (1.18×, 18%). Best-of-N never climbs meaningfully, producing correct kernels for 13 of 50 benchmarks but only beating XLA on one.

Iterative+context lands *more* correct kernels than Autocomp (48/50 vs. 45/50) yet reaches a lower geomean speedup. The two methods spend their 144-sample budget differently: iterative+context debugs until nearly every chain is correct, so it eventually solves 3 of the 5 benchmarks on which Autocomp generates no correct Pallas. Autocomp commits 4 translation cycles up front and then spends its remaining budget on optimization rather than debugging, which enables higher speedups. Per-benchmark best speedups are reported in Figure 6 (Appendix A).

Failure analysis. To understand *where* each method spends its sample budget, we classify every sample by outcome (Figure 4). Adding TPU-specific documentation sharply reduces API misuse. Iterative refinement with Autocomp's context raises per-sample correctness from 5.8% to 37.3% with no change to the search algorithm. Pallas API misuse dominates context-free failures, with 99.7% of best-of-N and 93.8% of iterative samples failing at compile or first execution by hallucinating Pallas APIs or misusing `pallas_call` arguments and `BlockSpec` types. API misuse remains a challenge even with context, comprising 59.8% of iterative+ctx and 55.8% of Autocomp samples.

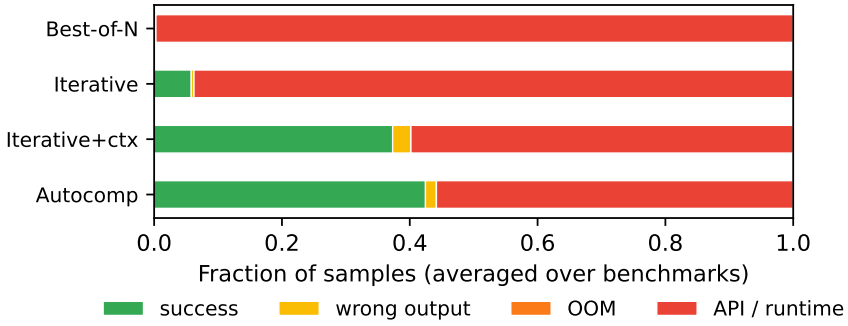


Figure 4 | Per-sample outcome breakdown on JAXBench (Gemini 3 Flash), pooling all samples across the 50 benchmarks. "Wrong output" denotes samples that compile and run but fall outside numerical tolerance. "API/runtime" covers unsupported Pallas API syntax and compile/runtime exceptions.

3.3. Comparison Against Hand-Tuned Pallas

Eight of the 17 priority operators ship with hand-optimized Pallas implementations from the upstream JAX Pallas ops library (Tokamax), with block sizes tuned via grid search on TPU v6e. We measured these kernels through the same JAXBench harness and report per-kernel speedups in Table 3. Tokamax reaches a 2.08× floor-1 geomean over the 8 kernels (the one sub-1× entry is a Splash Attention variant tuned for a different sparsity pattern than our dense-mask workload). Autocomp's 1.60× geomean on the same subset reaches roughly 77% of the hand-tuned geomean, exceeding Tokamax on 2 kernels and landing within 68–91% on 4 more, but failing correctness on the paged- and ragged-attention operators where Tokamax's hand-written scheduling matters most. Iterative+context with the same sample budget reaches 1.34×.

Table 3 | Speedups on the 8 priority kernels with hand-tuned Pallas references. Agents use a 144-sample Gemini 3 Flash budget. "—" denotes no correct kernel. Bold marks the best method per row.

Benchmark	XLA (ms)	HT (ms)	Hand-tuned	Autocomp	Iter+ctx	Iter
Flash Attention	25.21	6.45	**3.91**×	2.67×	0.46×	—
GQA Attention	51.13	14.59	**3.50**×	2.63×	2.63×	—
MLA Attention	20.32	21.72	0.94×	0.85×	**1.54**×	—
Sparse Attention	5.95	6.89	0.86×	**2.81**×	2.54×	—
Paged Attention	7.82	3.41	**2.29**×	—	0.61×	—
Ragged Paged Attention	18.70	2.71	**6.91**×	—	—	—
GEMM	5.36	5.59	0.96×	0.75×	**0.97**×	0.59×
Megablox GMM	3.26	2.01	1.62×	**2.21**×	0.27×	0.52×
Geomean (floor 1×)			**2.08**×	1.60×	1.34×	1.00×

3.4. Model Capacity Ablation (Gemini 3.1 Pro)

To measure the effect of model capacity, we rerun all four methods on a 5-kernel subset using Gemini 3.1 Pro, limited in scope due to Pro's substantially higher per-sample cost. The Flash curves in this ablation are remeasured on the same subset so that Flash and Pro are compared on identical benchmarks and sample budgets. Pro lifts every method, with the largest gains on iterative refinement, iterative+context, and Autocomp. Iterative jumps from 1.07× to 2.43×, iterative+context from 1.59× to 3.82×, and Autocomp from 2.35× to 3.79× (Figure 5). Pro iterative+context and Autocomp land correct Pallas on all 5 benchmarks, whereas Flash Autocomp still fails on Mamba-2 SSD.

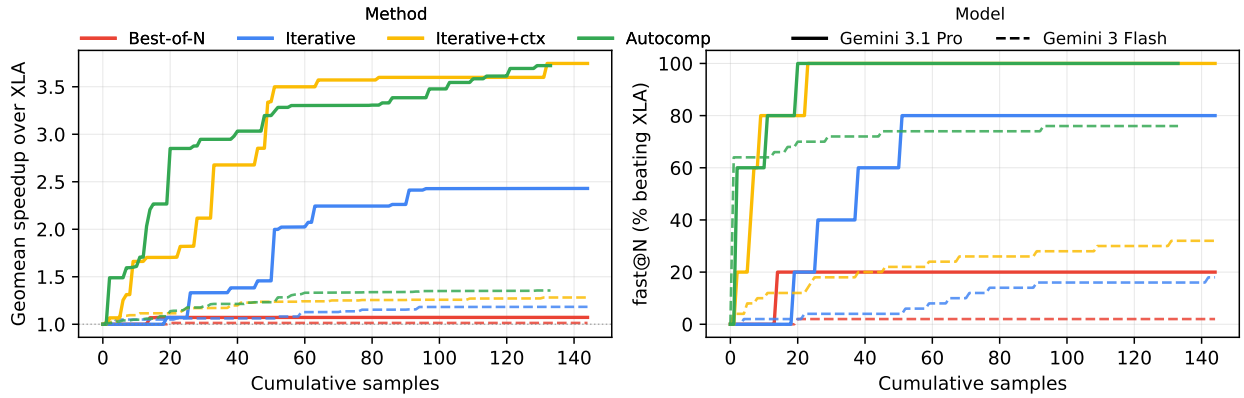


Figure 5 | Model-capacity ablation on the 5-kernel subset (RMSNorm, Flex Attention, RetNet Retention, Mamba-2 SSD, Flash Attention): speedup over XLA (left) and $\text{fast}_1@N$ (right) as a function of cumulative samples. Solid lines are Gemini 3.1 Pro; dashed lines are Gemini 3 Flash.

Plain iterative refinement benefits disproportionately from model scaling (1.07× → 2.43×) purely from better Pallas priors, and once both context and capacity are engaged, iterative+context and Autocomp converge to essentially the same geomean (3.82× vs. 3.79×). This is unlike the full Flash suite where Autocomp leads by a wide margin, since once Pro produces correct seeds without extensive debugging, iterative+context's samples go toward optimization rather than correctness recovery. On the two kernels where Autocomp Pro trailed iterative+context (Mamba-2 SSD and Flash Attention), Autocomp's best latency was still improving monotonically through the final iteration, so the per-kernel ordering reflects exploration depth at a fixed budget rather than beam-search convergence. We treat this narrowing as a preliminary observation given the 5-kernel scope.

4. Discussion and Future Work

Compiler, runtime, and profiler feedback alone are insufficient to drive performance. Iterative refinement without TPU-specific documentation achieves only 16.9% per-sample correctness on the 5-kernel subset with Gemini 3.1 Pro, and 1.2% with Flash. The Pallas constraints that govern correctness, including lexicographic grid traversal, VMEM/SMEM/HBM placement, (8, 128) block divisibility, and prefetch scheduling, are absent from error messages, leaving feedback loops nothing to iterate against.

Using a larger model helps, but not as much as adding useful context. Autocomp-generated documentation injected into the iterative loop raises per-sample correctness from 5.8% to 37.3% across the full Flash suite (Section 3.2), a much larger jump than the Flash-to-Pro gain at fixed context. For Pallas and other languages underrepresented in training data, the correctness bottleneck is information rather than reasoning, and written documentation supplies most of what is missing.

Once correctness is achieved, performance depends on how the sample budget is spent. Iterative refinement with only documentation solves *more* benchmarks than Autocomp on the full Flash suite (48/50 vs. 45/50) but rarely pushes past the first correct kernel, finishing at 1.28× geomean. Autocomp's translate/optimize split shifts the beam into optimization as soon as a correct seed exists, reaching 1.36×. Search structure, not just documentation, converts correctness into performance, though the Pro ablation shows this gap narrows once the model is strong enough to produce correct seeds without extensive debugging (Section 3.4).

Limitations. All 50 workloads run on a single TPU v6e chip, and multi-chip sharding and collective communication are out of scope. Hand-tuned Pallas references are available for 8 of the 17 priority operators and serve as an aspirational upper bound. The Gemini 3.1 Pro evaluation is restricted to a 5-kernel subset and is reported as a controlled model-capacity ablation rather than a primary result.

Future work. The most consequential extension is supporting multi-TPU kernel evaluation and optimization. Production training and inference workloads almost universally run across many chips, where end-to-end performance depends on sharding strategy, collective scheduling, and the overlap of communication with compute. Future work will extend JAXBench along the parallelism axis, covering tensor-parallel, pipeline-parallel, and expert-parallel sharding regimes on multi-chip TPU pods. A benchmark with these characteristics would evaluate whether the documentation-versus-search separation we observe generalizes to collective primitives (`shard_map`, `psum_scatter`, `all_to_all`, asynchronous collectives, and overlapping pipelines), which are even more sparsely represented in pretraining data than Pallas itself. More broadly, scaling AI-for-systems methods from single-device benchmarks to real-world deployment surfaces recurring challenges around topology-aware scheduling, communication overhead, and reproducibility of measured speedups [28].

Beyond multi-chip, the information-gap result motivates fine-tuning or reinforcement learning from execution feedback on curated Pallas corpora, for which JAXBench correctness and latency signals are usable as rewards. The tier structure in our breakdown also suggests hybrid controllers that route between iterative refinement and beam-search agents on a per-benchmark basis. We release the benchmark, baselines, and harness to support these directions.

5. Conclusion

Hand-optimized accelerator kernels remain a persistent bottleneck for ML systems, and the pool of engineers fluent in both ML and low-level systems programming cannot keep pace with new architectures and hardware revisions. Recent benchmarks show that LLMs can begin to fill this gap on CUDA and Triton, but the same machinery fails on TPUs because Pallas is underrepresented in training data. JAXBench helps close the measurement gap with 50 production-scale JAX workloads, 8 hand-tuned Pallas references with grid-searched block sizes, and a profiler-grounded evaluation harness on TPU v6e.

Across the suite, automated methods make real progress but still trail expert kernels. Methods conditioned on documentation, like iterative refinement and Autocomp's beam search, beat the baseline XLA performance on many workloadse. The dominant lever on niche backends is target-specific context rather than model scale or feedback alone, and search structure is what converts correctness into performance once that information gap is closed. We release the suite, baselines, and harness to support evaluation for current methods and set a foundation for the multi-chip and training-time work to come.

Acknowledgements

We thank Alain Hamel, Newfel Harrat, Steven Ingram, Mehrdad Khani, Gerson Kroiz, and Tomas Pfister for their helpful feedback and support throughout this work.

References

[1] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, et al. Jax: Autograd and xla. *Astrophysics Source Code Library*, pages ascl–2111, 2021.

[2] Shiyi Cao, Ziming Mao, Joseph E Gonzalez, and Ion Stoica. K-search: Llm kernel generation via co-evolving intrinsic world model. *arXiv preprint arXiv:2602.19128*, 2026.

[3] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.

[4] Google AI Hypercomputer. MaxText: A high performance, scalable, open-source LLM in pure Python/JAX. https://github.com/AI-Hypercomputer/maxtext, 2022. Accessed: 2026-04-26.

[5] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[6] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[7] Charles Hong, Sahil Bhatia, Alvin Cheung, and Sophia Shao. Autocomp: Llm-driven code optimization for tensor accelerators. In *Machine Learning for Computer Architecture and Systems 2025*, 2025.

[8] JAX Development Team. Writing TPU kernels with pallas. https://docs.jax.dev/en/latest/pallas/tpu/details.html, 2024. Accessed: 2026-04-26.

[9] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[10] Jevin Jiang, Ying Chen, Blake A Hechtman, Fenghui Zhang, and Yarong Mu. Ragged paged attention: A high-performance and flexible llm inference kernel for tpu. *arXiv preprint arXiv:2604.15464*, 2026.

[11] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*, pages 1–12, 2017.

[12] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with alphafold. *nature*, 596(7873):583–589, 2021.

[13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.

[14] Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. Towards robust agentic cuda kernel benchmarking, verification, and optimization. *arXiv preprint arXiv:2509.14279*, 2025.

[15] Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, WangHaojie WangHaojie, Jianrong Wang, Xu Han, et al. Tritonbench: Benchmarking large language model capabilities for generating triton operators. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 23053–23066, 2025.

[16] Gang Liao, Hongsen Qin, Ying Wang, Alicia Golden, Michael Kuchnik, Yavuz Yetim, Jia Jiunn Ang, Chunli Fu, Yihan He, Samuel Hsia, et al. Kernelevolve: Scaling agentic kernel coding for heterogeneous ai accelerators at meta. *arXiv preprint arXiv:2512.23236*, 2025.

[17] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[18] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

[19] OpenXLA Contributors. Tokamax: A GPU and TPU kernel library. https://github.com/openxla/tokamax, 2024. Accessed: 2026-04-26.

[20] Anne Ouyang, Simon Guo, Simran Arora, Alex L Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can llms write efficient gpu kernels? *arXiv preprint arXiv:2502.10517*, 2025.

[21] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[22] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[23] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

[24] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[25] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Aditya Atluri, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. CUTLASS. https://github.com/NVIDIA/cutlass, 2023. Accessed: 2026-04-26.

[26] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

[27] Arya Tschand, Muhammad Awad, Ryan Swann, Kesavan Ramakrishnan, Jeffrey Ma, Keith Lowery, Ganesh Dasika, and Vijay Janapa Reddi. Swizzleperf: Hardware-aware llms for gpu kernel performance optimization. *arXiv preprint arXiv:2508.20258*, 2025.

[28] Arya Tschand, Chenyu Wang, Zishen Wan, Andrew Cheng, Ioana Cristescu, Kevin He, Howard Huang, Alexander Ingare, Akseli Kangaslahti, Sara Kangaslahti, Theo Lebryk, Hongjin Lin, Jeffrey Jian Ma, Alexandru Meterez, Clara Mohri, Depen Morwani, Sunny Qin, Roy Rinberg, Paula Rodriguez-Diaz, Alyssa Mia Taliotis, Pernille Undrum Fathi, Rosie Zhao, Todd Zhou, and Vijay Janapa Reddi. Genai for systems: Recurring challenges and design principles from software to silicon, 2026. URL https://arxiv.org/abs/2602.15241.

[29] Zhongzhen Wen, Yinghui Zhang, Zhong Li, Zhongxin Liu, Linna Xie, and Tian Zhang. Multikernelbench: A multi-platform benchmark for kernel generation. *arXiv e-prints, pp. arXiv–2507*, 2025.

[30] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[31] Shanli Xing, Yiyan Zhai, Alexander Jiang, Yixin Dong, Yong Wu, Zihao Ye, Charlie Ruan, Yingyi Huang, Yineng Zhang, Liangsheng Yin, et al. Flashinfer-bench: Building the virtuous cycle for ai-driven llm systems. *arXiv preprint arXiv:2601.00227*, 2026.

[32] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in neural information processing systems*, 32, 2019.

A. Per-Benchmark Results

Figure 6 shows the best speedup achieved by each method on every benchmark in the 50-workload JAXBench suite with Gemini 3 Flash. Benchmarks are sorted by the best speedup achieved by any method, and missing markers indicate the method produced no correct implementation within its sample budget.

Table 4 reports the corresponding per-benchmark best speedups for the Gemini 3.1 Pro ablation on the 5-kernel subset (Section 3.4).

Table 4 | Per-benchmark best speedup over XLA on the 5-kernel subset with Gemini 3.1 Pro. "—" denotes no correct kernel within the 144-sample budget. Bold marks the best method per row. Geomean floors per-kernel speedups at 1× before aggregating.

Benchmark	XLA (ms)	Best-of-*N*	Iter	Iter+ctx	Autocomp
RMSNorm	1.24	1.42×	1.42×	1.43×	**1.44**×
Flex Attention	36.81	—	3.99×	3.46×	**4.01**×
RetNet Retention	13.04	—	4.85×	6.40×	**9.62**×
Mamba-2 SSD	29.59	—	—	**5.66**×	4.04×
Flash Attention	25.38	—	3.07×	**4.54**×	3.51×
Geomean (floor 1×)		1.07×	2.43×	**3.82**×	3.79×

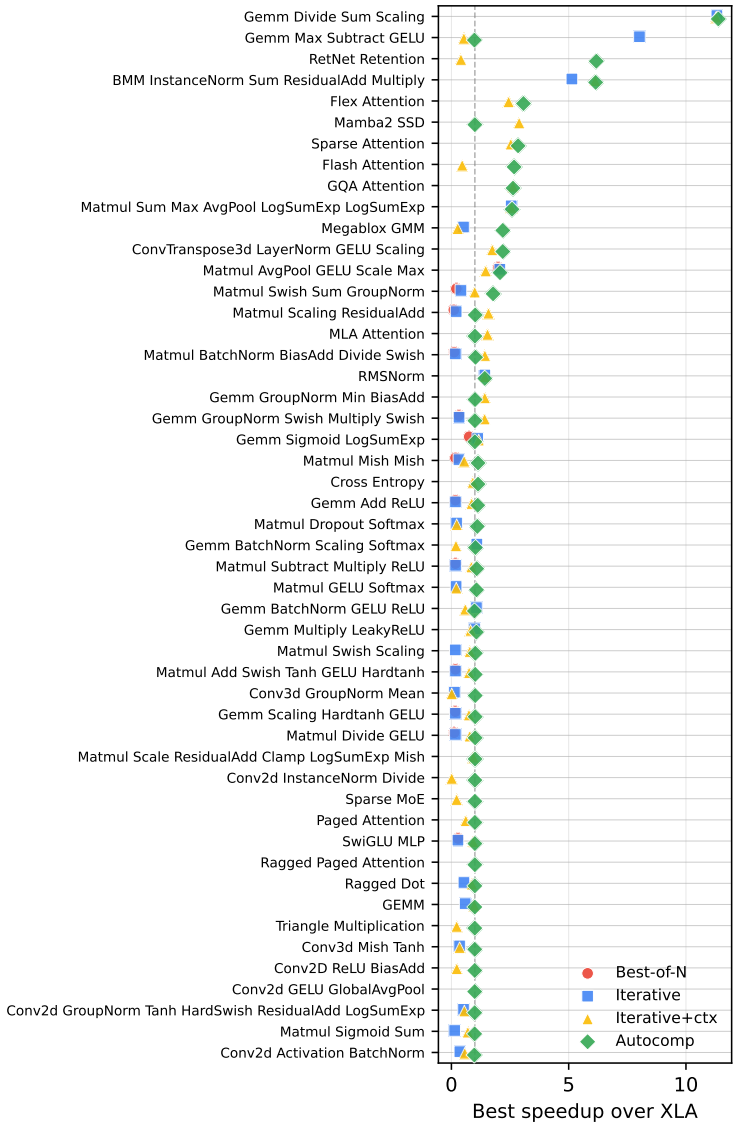


Figure 6 | Best speedup over the XLA baseline for each benchmark and method. Benchmarks without a marker had no correct implementation within the sample budget.

B. Baseline Prompts and Agent Context

This appendix documents the exact prompt construction for each evaluated method so the setup can be reproduced from the paper text alone.

B.1. Best-of-N and Iterative Refinement

Both baselines use the same short system preamble:

```
You are optimizing a JAX kernel for TPU v6e (Trillium) using the Pallas programming
model (jax.experimental.pallas).
Target hardware:  TPU v6e.  VMEM = 128 MiB, 8 TensorCores per chip, peak ≈918
TFLOPS bf16, HBM ≈1640 GB/s.
Rules for every turn:  (1) keep the public workload(*inputs) signature unchanged;
(2) output a single complete Python file inside one "‘python ..."‘ block; (3)
```

```
do not include explanatory prose outside the code block.
Strategy:  if you do not yet have a correct Pallas implementation, your first
priority is to produce a correct, straightforward translation even if it isn't
faster than the XLA baseline.  Only after you have a correct Pallas kernel
should you focus on optimizing it for speed.
```

Best-of-N. Each of the N samples is an independent completion of a prompt consisting of the preamble above, a single line `Benchmark: <prob_id>`, a one-sentence task description ("Below is the XLA/JAX reference implementation. Rewrite it as a Pallas kernel that produces identical outputs and runs faster."), and the JAX source wrapped in a Python code block. No feedback is provided.

Iterative refinement. The first turn of each chain uses the same prompt as Best-of-N. Each subsequent turn within a chain appends the following feedback block, then asks the model for a new complete Pallas implementation:

- If the previous attempt was correct: "Previous attempt was correct. Latency: <x> ms (XLA baseline: <y> ms, speedup: <s> ×). Best-so-far in this chain: <z> ms."
- If the previous attempt was incorrect: "Previous attempt was incorrect or failed to run." followed by the last 40 lines of the runner's stdout.
- A profiler summary (top operators by time) from the previous run when available.
- The previous implementation's full source, wrapped in a Python code block.

Chains run independently and never share context, and 18 chains of 8 turns give a total of 144 samples per benchmark.

Iterative refinement with Autocomp context. The *iterative+context* variant uses the same chain/turn/feedback layout as plain iterative refinement (18 chains × 8 turns, same feedback block on correctness or failure, same final-code echo) with one change. Before every prompt we prepend the static portion of Autocomp's agent context, namely the hardware architecture summary, the per-benchmark-selected Pallas API reference, the selected code examples, and the rules block (see next subsection). The per-benchmark category and example selections are made by the same LLM yes/no filter Autocomp uses and are cached per benchmark, so a chain's eight turns see the same context menu. The variant does not use Autocomp's beam search or translate/optimize phase split, and is purely a context injection into the iterative baseline.

B.2. Autocomp Context Artifacts

We passed four JAX and Cloud TPU documentation sources into Autocomp's Agent Builder pipeline:

- JAX Pallas guide: https://docs.jax.dev/en/latest/pallas/index.html
- JAX Pallas TPU guide: https://docs.jax.dev/en/latest/pallas/tpu/index.html
- `jax.experimental.pallas.tpu` API reference: https://docs.jax.dev/en/latest/jax.experimental.pallas.tpu.html
- Cloud TPU documentation: https://docs.cloud.google.com/tpu/docs/

Each source is crawled with `max_depth=2` and `max_pages=250`. An LLM-driven routing-and-synthesis pass then distills the crawled content into four textual artifacts that together form the static portion of every agent prompt:

- **Hardware architecture summary** (≈8 KB): a single-pass LLM summary of TPU v6e characteristics (memory hierarchy, compute units, bandwidth and capacity numbers, and programming-model constraints).
- **Pallas API / ISA reference** (≈115 KB): per-API entries (signature, description, parameter semantics, inline usage snippets) extracted verbatim from the source documentation and grouped into functional categories. At prompt time, an LLM yes/no filter selects which categories are relevant to the current kernel and caches the selection per-benchmark.
- **Annotated code examples** (≈30 KB): representative Pallas kernels drawn from the ingested documentation, each accompanied by a one-line summary. Examples are selected per-benchmark via a parallel LLM yes/no filter.
- **Rules** (≈6 KB): correctness constraints (e.g., block-shape divisibility, memory-space annotations, aliasing rules) extracted as imperative bullet points and appended to every planning and coding prompt.

During beam search, each candidate-generation prompt concatenates the architecture summary, the per-benchmark-selected API reference, any selected code examples, the candidate's current code (plus its latest score and any hardware-counter feedback), and the rules block, before asking the model for either a plan or a complete optimized kernel. The translate and optimize phases use the same scaffold but draw from separate menus of strategies.